

Methods in Python

22 June 2026 10:59

In Python a method is a function that belongs to a specific object or class. While a regular function is called independently by its name (like `print()`) a method is always invoked on object using dot notation (like `Object.method()`).

```
p = m.exp2(4)
q = m.exp(2)
print(f'e={m.e} to the power of 2 = {q}')
print(p)
```

Output:

```
e=2.718281828459045 to the power of 2 = 7.38905609893065
16.0
```

Method allows object to perform action on their own internal data, forming the backbone of object oriented programming (OOP).

In Python, a method is a function that belongs to an object or a class. Methods are used to perform operations on data associated with that object.

Difference between function and method

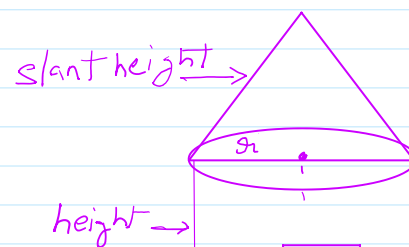
Function	Method
Independent block of code.	Function associated with an object or class.
Called directly.	Called through an Object
Example: <code>print()</code>	Example: <code>math.exp()</code>

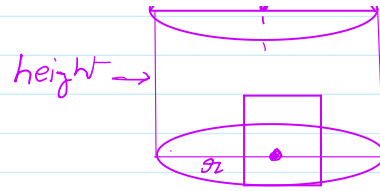
Why use Methods?

Method helps to:

- Organize code
- Reuse code
- Perform operations on objects
- Implement Object-Oriented Programming(OOP).

In programming, the use of function is one of the means to achieve modularity and reusability. Function can be defined as a named group of instructions that accomplish a specific task when it is invoked. Once defined, a function can be called repeatedly from different places of the program without writing all the code of the that function every time Or it can be called from inside another function by simply writing the name of the function and passing the required parameters if any. The programmer can define as many function as desired while writing the code.





```
# Program to calculate the cost of tent
#
import math
# function definition
def areaOfCyl(h,r):
    area_cyl = 2*math.pi*r*h
    return area_cyl
# Function definition
def areaOfCone(l, r):
    area_con = math.pi*r*l #Area of conical part
    return area_con

# function definition
def post_tax_price(cost):
    tax = 0.18 * cost
    net_price = cost + tax
    return net_price

# main code
h = float(input("Height of cylinder: "))
r = float(input(" Enter radius of cylinder: "))
area_of_cylinder = areaOfCyl(h, r)
l = float(input(" Enter slant height of the conical area in meters : "))
conical_area = areaOfCone(l, r)
canvas_area = conical_area + area_of_cylinder
print(f"Area of canvas: {canvas_area:.2f} m^2")
#calculate the cost of canvas
unit_price = float(input(" Inter cost of 1m square Canvas in rupees: \u20b9 "))
total_cost = unit_price * canvas_area
print(f"The total cost of canvas before tax equal to \u20b9:{total_cost:.2f}")
print(f"Net amount payable(including tax) = \u20b9{post_tax_price(total_cost):.2f}")
```

Output:

```
Height of cylinder: 3
Enter radius of cylinder: 1.5
Enter slant height of the conical area in meters : 1.5
Area of canvas: 35.34 m^2
Inter cost of 1m square Canvas in rupees: ₹ 50
The total cost of canvas before tax equal to ₹:1767.15
Net amount payable(including tax) = ₹ 2085.23
```

Home work

1. Detailed Tent Inventory and Volume Analyzer

Extend the procedural code from the image. Write a program that uses functions to calculate both the surface area and the internal volume of a composite tent

structure.

- Create functions `volume_of_cylinder(h, r)` and `volume_of_cone(h_cone, r)`. Derive the vertical height of the cone from the slant height `l` and radius `r` using the math equation: $\text{slant_height_cone} = \sqrt{l^2 - r^2}$.
- Print the Canvas Area, Internal Volume, and Total Material Cost.

2. Tiered Discount and Multi-State Tax Calculator

In the script, a flat tax of 18% is applied using a simple function. Upgrade this billing flow into a flexible function named `calculate_final_bill(bulk_cost, state_code)`.

- Apply a **tiered discount** to the raw canvas cost before adding tax: 5% discount if the cost is above ₹5,000, 10% if above ₹10,000, and 15% if above ₹20,000.
- Apply different tax rates depending on a string input `state_code` (e.g., "ST1" = 12%, "ST2" = 18%, "ST3" = 28%).

3. Tent Structure Waste Material Estimator

When cutting raw canvas sheets to manufacture a tent, a percentage of material is wasted during production.

- Use the exact functions `areaOfCyl(h, r)` and `areaOfCone(l, r)` from the image text.
- Write a main program logic block that prompts the user for a `waste_percentage` integer (e.g., 12% waste).
- Calculate the total raw canvas area that must actually be purchased to account for this production overhead.
- Print the exact monetary cost difference caused strictly by the wasted material using formatted f-strings.

Syntax of a User-Defined Function

Keyword used to define a function → *Name of the function*

```
def function_name(parameters):
```

Values accepted by the function

```
    """Optional document string"""
```

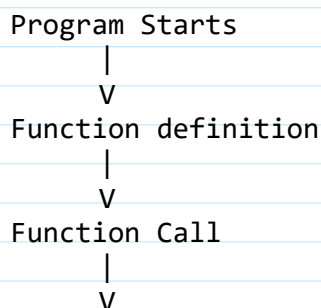
Function body → *Statements executed when the function is called*

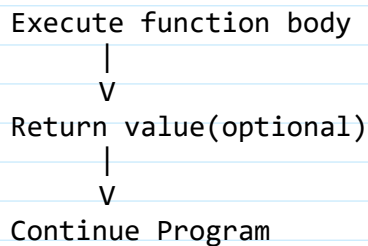
```
    Statements
```

Returns a value to the caller (optional)

```
    return value
```

Flow of Function or Method Execution





```
def greet(name):  
    print(f'Hello! {name}')
```

main code
nm = input('Enter your name: ')
greet(nm)

Types of User-Defined Functions

Functions can be classified due to four categories.

Type 1: No argument and no return value.

Example:

```
def greet(name): → Type-2  
    print(f'Hello! {name}')
```

```
def welcome(): → Type-1  
    print('*****Welcome to canvas shop*****')
```

main code
nm = input('Enter your name: ')
greet(nm)
welcome()

Type 2: Argument but No Return Value

Type 3: No Argument but Return Value

Type 4: Arguments and Return value

```
# Program to calculate the cost of tent  
#  
import math  
# function definition  
def areaOfCyl(h,r):  
    area_cyl = 2*math.pi*r*h  
    return area_cyl
```

Function definition
def areaOfCone(l, r):
 area_con = math.pi*r*l #Area of conical part
 return area_con

function definition
def next_tax_price(cost):

Type 4: Arguments & return value.

```

# function definition
def post_tax_price(cost):
    tax = 0.18 * cost
    net_price = cost + tax
    return net_price

def greet(name):
    print(f'Hello! {name}')

def welcome():
    print('*****Welcome to canvas shop*****')

def getHeight():
    h = float(input("Height of cylinder: "))
    return h

def getRadius():
    r = float(input(" Enter radius of cylinder: "))
    return r

def getSlantLength():
    l = float(input(" Enter slant height of the conical area in meters : "))
    return l

# main code
nm = input('Enter your name: ')
greet(nm)
welcome()
h = getHeight()
r = getRadius()
area_of_cylinder = areaOfCyl(h, r)
l = getSlantLength()
concial_area = areaOfCone(l, r)
canvas_area = concial_area + area_of_cylinder
print(f"Area of canvas: {canvas_area:.2f} m^2")
#calculate the cost of canvas
unit_price = float(input("Enter cost of 1m square Canvas in rupees: \u20b9 "))
total_cost = unit_price * canvas_area
print(f"The total cost of canvas before tax equal to \u20b9:{total_cost:.2f}")
print(f"Net amount payable(including tax) = \u20b9{post_tax_price(total_cost):.2f}")

```

Type-3 : No argument but return value.

Function with Multiple Parameters

```

def student(name, age, course):
    print(f'Name: {name}')
    print(f'Age: {age}')
    print(f'Course: {course}')

#main code
nm = input('Enter name: ')
age = int(input('Enter age: '))

```

```
course = input('Enter course name: ')
student(nm, age, course)
```

Function Returning Multiple Values

Python allows multiple values to be returned.

→ local variable to the function.

```
def calculate(a, b):
    return a+b, a-b, a*b, a/b, a//b, a**b, a%b
#main code
a = float(input('Enter any number: '))
b = float(input('Enter any number: '))
sum, diff, prod, div, quo, exp, rem = calculate(a, b)
print(f'sum = {sum}')
print(f'difference = {diff}')
print(f'product = {prod}')
print(f'quotient = {quo}')
print(f'exponent = {exp}')
print(f'remainder = {rem}')
```

Output:

```
Enter any number: 8
Enter any number: 3
sum = 11.0
difference = 5.0
product = 24.0
quotient = 2.0
exponent = 512.0
remainder = 2.0
```

Local variables: Variables created inside of functions are called local variables.

```
def f1():
    x = 100
    print(x)
    x += 20
    print(x)
```

→ Local variable in method f1(), not accessible outside f1().

```
#main code
f1()
```

```
def f1():
    x = 100
    print(x)
    x += 20
```

is not a local variable.

"x" is not defined Pylance(reportUndefinedVariable)

(variable) x: Any

View Problem (Alt+F8) Quick Fix... (Ctrl+.) Fix (Ctrl+I)

```
x += 10
```

Global Variables

Variables declared outside all functions are called global variables.

```

msg = 'Hello! Python'
def f1():
    x = 100
    print(x)
    x += 20
    print(x)
    print(f'from f1(): {msg}')

#main code
f1()
# x += 10
print(f'From main: {msg}')

```

Output:

```

100
120
from f1(): Hello! Python
From main: Hello! Python

```

Using the global Keyword

```

x = 500
def f1():
    x = 100
    print(f'Local x = {x}')
    x += 20
    print(f'Local x = {x}')

#main code
f1()
x += 10
print(f'Global x = {x}')

```

Output:

```

Local x = 100
Local x = 120
Global x = 510

```

```

x = 500
def f1():
    global x
    x = 100
    print(f'Global x = {x}')
    x += 20
    print(f'Global x = {x}')

#main code
f1()
x += 10
print(f'Global x = {x}')

```

—————> If global keyword is not used then 'x' variable is a local variable defined inside the method.

Output:

```
Global x = 100
Global x = 120
Global x = 130
```

Positional Arguments

Arguments are passed according to their position.

```
def details(name, city):
    print(name)
    print(city)

details("Amit", "Delhi")
```

Output:

```
Amit
Delhi
```

Keyword Arguments

Arguments are passed using parameter names.

```
def details(name, city, pincode):
    print(f'Name: {name}')
    print(f'City: {city}')
    print(f'Pincode: {pincode}')

details("Amit", "Delhi", 700011)
# Keyword arguments
details(pincode=700084, name='Shiv', city='Kolkata')
```

Output:

```
Name: Amit
City: Delhi
Pincode: 700011
Name: Shiv
City: Kolkata
Pincode: 700084
```

Default Arguments

A default value is assigned to a parameter.

```
def details(name, city = 'Kolkata', pincode=700001):
    print(f'Name: {name}')
    print(f'City: {city}')
    print(f'Pincode: {pincode}')

details("Amit", "Delhi", 700011)
# Keyword arguments
details(pincode=700084, name='Shiv', city='Kolkata')
details('Sankar')
details('Yash', 'Mumbai')
```

Output:

```
Name: Amit
```



```
City: Delhi
Pincode: 700011
Name: Shiv
City: Kolkata
Pincode: 700084
Name: Sankar
City: Kolkata
Pincode: 700001
Name: Yash
City: Mumbai
Pincode: 700001
```

Variable-length Arguments(*args)

Allows multiple values to be passed.

```
def total(*numbers):
    print(sum(numbers))
```

```
total(23, 45)
total(6, 19, 2)
total(10, 30, 20, 50)
```

Output:

```
68
27
110
```

Keyword Variable-Length Arguments(**kwargs)

Accept multiple named argument.

```
def student(**details):
    return details

stu1 = student(name = 'Rahul', age=21, city= 'Delhi')
stu2 = student(name = 'Shiv', age=21, city= 'Bhopal')
stu3 = student(name = 'Raj', age=21, city= 'Varanasi')
stu4 = student(name = 'Jhon', age=21, city= 'Kolkata', state="WB")
print(stu1)
print(stu2)
print(stu3)
print(stu4)
```

Output:

```
{'name': 'Rahul', 'age': 21, 'city': 'Delhi'}
{'name': 'Shiv', 'age': 21, 'city': 'Bhopal'}
{'name': 'Raj', 'age': 21, 'city': 'Varanasi'}
{'name': 'Jhon', 'age': 21, 'city': 'Kolkata', 'state': 'WB'}
```

Anonymous (Lambda) Functions

The Lambda function is a small function without a name.

```
square = lambda x: x**2
```

```
print(square(4))
print(square(10))
```

Output:

```
16
100
```

Modularized coding example using User Defined methods in Python

```
def add(a, b):
    return a + b
def subtract(a, b):
    return (a - b)
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return 'Divisible by zero is not allowed'
    else:
        return a / b
def remainder(a, b):
    if b == 0:
        return 'Divisible by zero is not allowed'
    else:
        return a % b
def inputValue():
    global a, b
    a = int(input('Enter first number: '))
    b = int(input('Enter second number: '))

#main code
a = 0
b = 0
while True:
    print('====Basic Calculator====')
    op = input('Enter any arithmetic operator (+,-,*,/,%,x(exit)): ')

    match op:
        case '+':
            inputValue()
            sum = add(a,b)
            print(f'sum = {sum}')
        case '-':
            inputValue()
            diff = subtract(a,b)
            print(f'difference = {diff}')
        case '*':
            inputValue()
            prod = multiply(a, b)
            print(f'Product = {prod}')
        case '/':
            inputValue()
            div = divide(a,b)
            print(f'Quotient = {div}')
```

```

case '%':
    inputValue()
    rem = remainder(a, b)
    print(f'Remainder = {rem}')
case 'x' | 'X':
    print('Good bye')
    exit()
case _:
    print('Wrong option is selected')

```

Output:

```

=====Basic Calculator=====
Enter any arithmetic operator (+,-,*,/,%,x(exit)): 0
Wrong option is selected
=====Basic Calculator=====
Enter any arithmetic operator (+,-,*,/,%,x(exit)): %
Enter first number: 10
Enter second number: 0
Remainder = Divisible by zero is not allowed
=====Basic Calculator=====
Enter any arithmetic operator (+,-,*,/,%,x(exit)): %
Enter first number: 10
Enter second number: 4
Remainder = 2
=====Basic Calculator=====
Enter any arithmetic operator (+,-,*,/,%,x(exit)): x
Good bye

```